6. Auditoría de Sistema Web

6.1. Descripción del sistema web y herramientas de análisis

Para la auditoría de las aplicaciones web que dan soporte a las actividades de la universidad se desplegó un entorno de laboratorio aislado sobre contenedores Docker, garantizando que la totalidad de las pruebas se ejecutara exclusivamente sobre sistemas controlados y nunca sobre infraestructura en producción, en cumplimiento de la cláusula de uso lícito del proyecto. El entorno se compone de dos aplicaciones deliberadamente vulnerables:

- **DVWA (Damn Vulnerable Web Application):** objetivo principal de la auditoría, publicada en `127.0.0.1:8080`. Permite seleccionar el nivel de seguridad, empleándose *low* para la fase de explotación y *high/impossible* para la verificación de las contramedidas.
- **OWASP Juice Shop:** aplicación de apoyo de arquitectura moderna (API REST y autenticación mediante JWT), publicada en `127.0.0.1:3001`.

Ambas aplicaciones se expusieron únicamente sobre la interfaz de *loopback*, de modo que no resultan accesibles desde la red externa. Como motor central de análisis dinámico (*DAST*) y proxy de interceptación se utilizó **OWASP ZAP 2.17**, apoyado por un perfil dedicado de Mozilla Firefox configurado para enrutar la totalidad de su tráfico, tanto HTTP como HTTPS, a través del proxy local situado en `127.0.0.1:8081`. La fase de *fuzzing* se alimentó de diccionarios de cargas propios para inyección SQL, *Cross-Site Scripting* y *Path Traversal*.

```
PS C:\Users\franr\JAVI\Universidad\26-27\Primer Cuatri\SSII\PAI\PAIS\PAI3\Evidencias_Tecnicas\04_Auditoria_Sistema_We> d
ocker compose ps
NAME                IMAGE                             COMMAND                  SERVICE     CREATED              STATUS
          PORTS
vulnaweb-dvwa       vulnerables/web-dvwa:latest       "/main.sh"               dvwa        About a minute ago   Up About a
 minute   127.0.0.1:8080->80/tcp
vulnaweb-juiceshop  bkimminich/juice-shop:latest      "/nodejs/bin/node /j…"   juiceshop   About a minute ago   Up About a
 minute   127.0.0.1:3001->3000/tcp
```

Figura 6.1: Laboratorio desplegado mediante Docker Compose (DVWA en el puerto 8080 y OWASP Juice Shop en el 3001), ambos contenedores en estado de ejecución sobre la interfaz de loopback.

6.2. Plan de pruebas y resultados

El plan de pruebas se estructuró en dos grandes líneas de actuación: en primer lugar, la demostración de la trazabilidad del tráfico cifrado a través del proxy interceptor sin alterar las peticiones; y en segundo lugar, la explotación controlada de vulnerabilidades del *OWASP Top 10 2025*, acompañada de su correspondiente plan de mitigación.

Configuración del entorno de interceptación

ZAP se configuró como proxy local escuchando en `localhost:8081`, y el perfil de pruebas de Firefox se dirigió a dicho proxy tanto para HTTP como para HTTPS. Para permitir la interceptación del tráfico cifrado sin alertas de certificado, se exportó la autoridad certificadora raíz dinámica de ZAP y se importó en el almacén de Autoridades de Firefox, marcándola como de confianza para la identificación de sitios web.

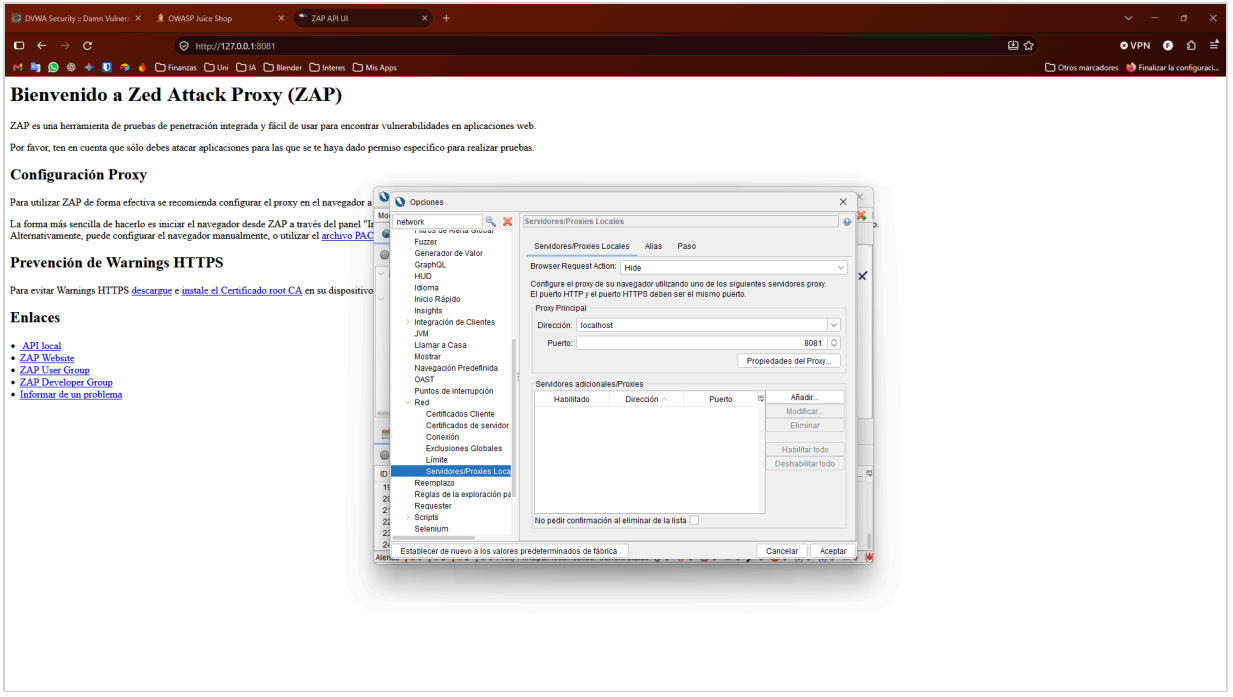


Figura 6.2: Configuración del proxy local de ZAP (localhost:8081) y enrutamiento del tráfico del navegador a través de dicho proxy.

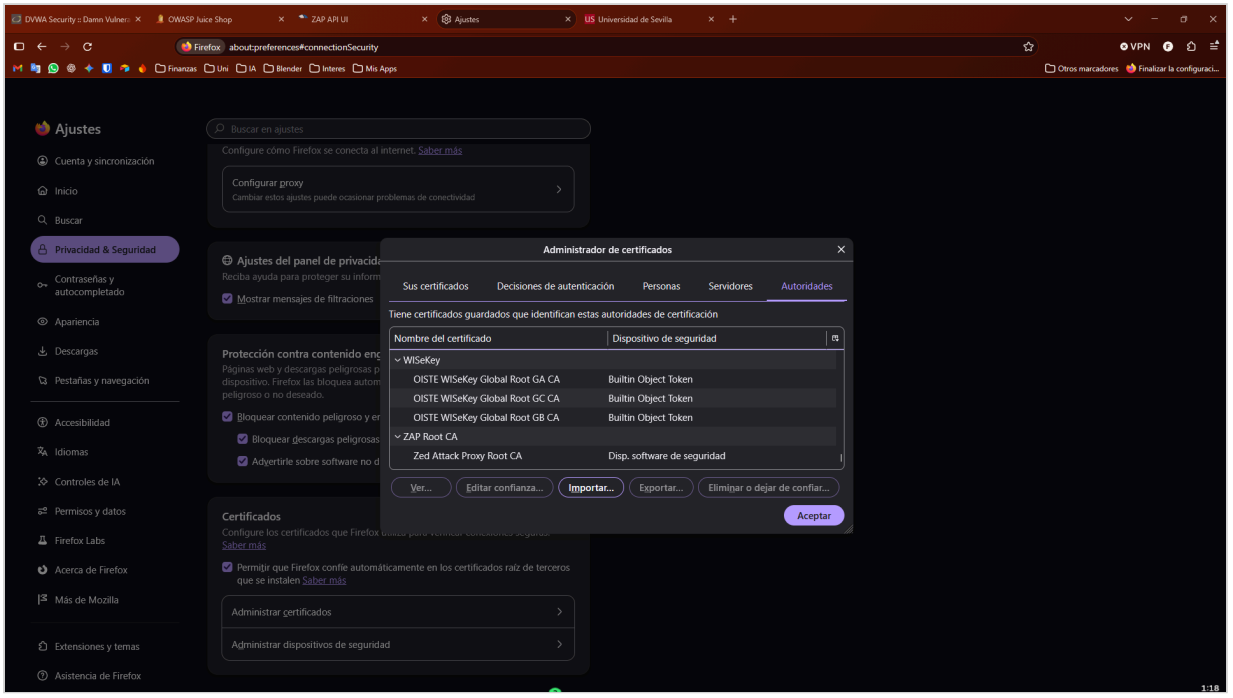


Figura 6.3: Autoridad certificadora raíz de ZAP (Zed Attack Proxy Root CA) importada y confiada en el almacén de Autoridades de Firefox.

Trazabilidad del tráfico HTTP/HTTPS sin modificación

Con ZAP operando en modo estándar, sin puntos de interrupción activos, se navegó por la aplicación de laboratorio sobre HTTP y por un sitio real sobre HTTPS. El panel *History* registra la totalidad de las peticiones y respuestas de forma íntegra, sin que se altere ningún parámetro, lo que satisface el requisito de trazabilidad exigido. La inspección del certificado del sitio HTTPS desde el navegador

confirma que éste ha sido reemitido por la CA de ZAP, evidenciando que el proxy intercepta y traza el tráfico cifrado de extremo a extremo.

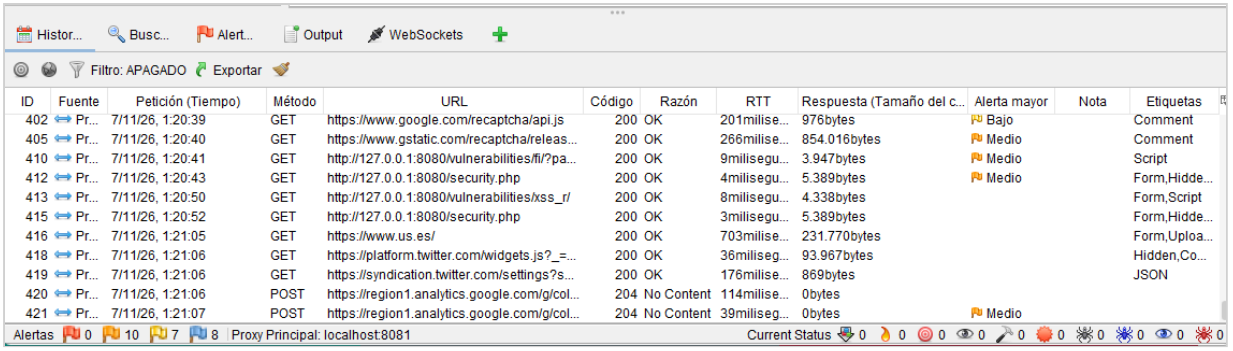


Figura 6.4: Panel History de ZAP mostrando la trazabilidad simultánea de peticiones HTTP (127.0.0.1:8080) y HTTPS (www.us.es) sin modificación alguna.

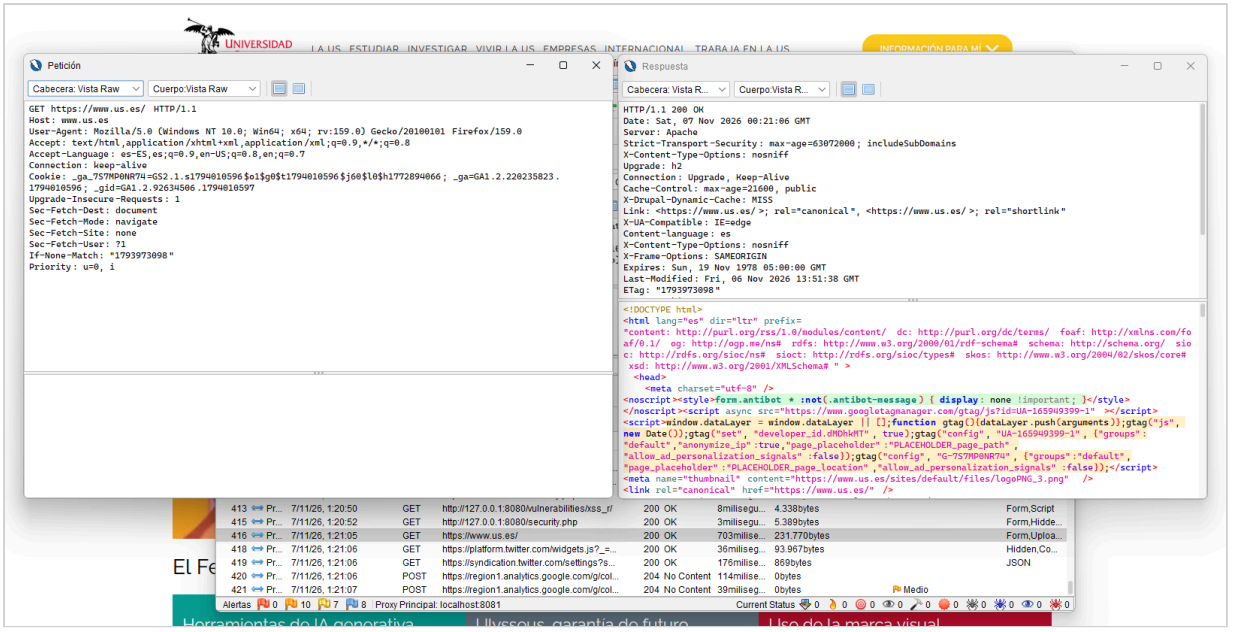


Figura 6.5: Petición y respuesta completas de una comunicación HTTPS trazada a través del proxy, mostradas de forma íntegra.

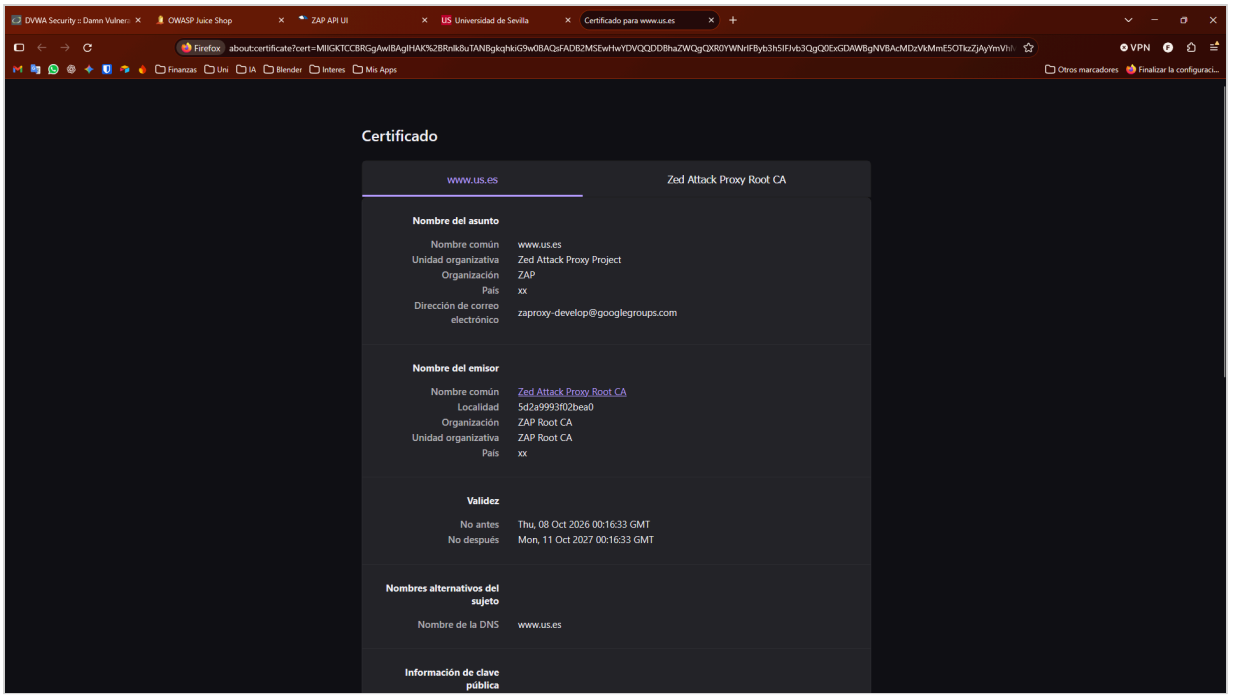


Figura 6.6: Certificado del sitio HTTPS reemitido dinámicamente por la autoridad raíz de ZAP (emisor: Zed Attack Proxy Root CA), prueba de la interceptación del tráfico cifrado.

Interceptación activa (Man-in-the-Middle)

Como demostración diferenciada de la trazabilidad, se activó un punto de interrupción (*breakpoint*) para capturar una petición POST de autenticación. ZAP detuvo la petición, permitiendo la alteración de sus parámetros, en particular el campo `password`, antes de reenviarla al servidor. Este escenario ilustra la capacidad del proxy para actuar como intermediario activo (*Man-in-the-Middle*), modificando el tráfico en tránsito.

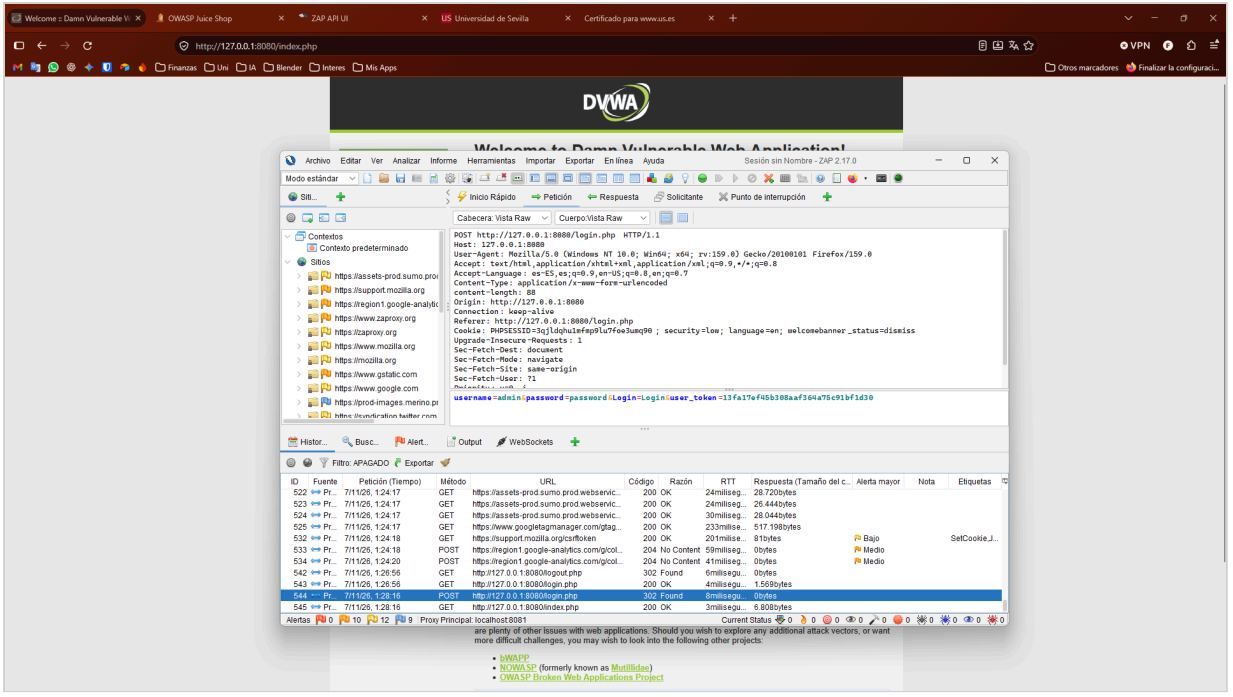


Figura 6.7: Petición POST de autenticación interceptada mediante breakpoint, con los parámetros de credenciales visibles y manipulables antes de su reenvío al servidor.

Explotación de vulnerabilidades y fuzzing automatizado

Sobre la aplicación de laboratorio se identificaron y explotaron, como mínimo, tres vulnerabilidades del OWASP Top 10 2025. Para cada una se partió de una petición legítima capturada, se aplicó el *Fuzzer* de ZAP con el diccionario correspondiente y se confirmó el hallazgo mediante prueba manual.

1. Inyección SQL (A05 – Injection)

Sobre el parámetro `id` del módulo *SQL Injection* de DVWA, el Fuzzer envió 26 peticiones a partir del diccionario `sqli.txt`. Las cadenas `' OR '1'='1` y `' OR '1'='1' #` se marcaron como *Reflected*, con un cuerpo de respuesta mayor que el de la petición base (4.821 y 4.836 bytes frente a 4.479), mientras que las variantes basadas en tiempo `1' OR SLEEP(5)` elevaron el tiempo de respuesta a 2–3 segundos, confirmando además la existencia de inyección ciega. La explotación manual mediante `' OR '1'='1` provocó el volcado de la totalidad de los usuarios, y la carga `' UNION SELECT user, password FROM users-- -` permitió extraer los usuarios *admin*, *gordonb*, *1337*, *pablo* y *smithy* junto a sus contraseñas almacenadas como hash MD5 sin salt (por ejemplo, `5f4dcc3b5aa765d61d8327deb882cf99`, correspondiente a la cadena «password»).

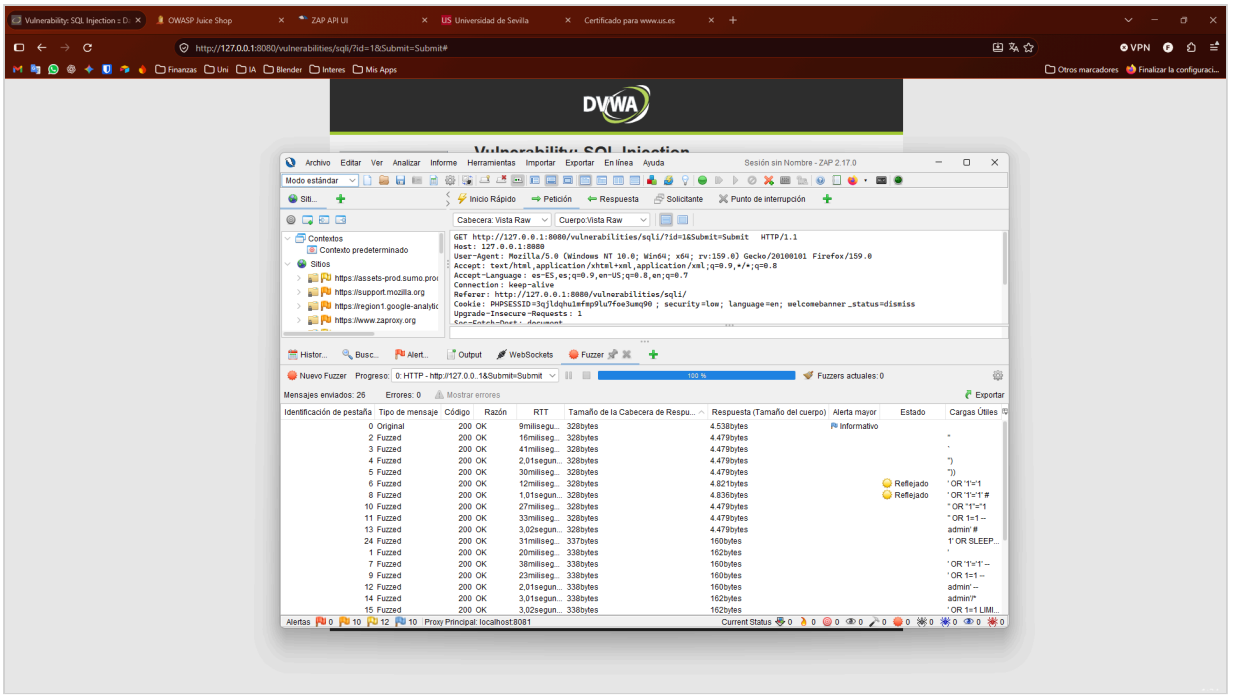


Figura 6.8: Resultados del Fuzzer de ZAP sobre el parámetro id, con las cargas marcadas como Reflected y las variantes basadas en tiempo (SLEEP) identificadas por su mayor tiempo de respuesta.

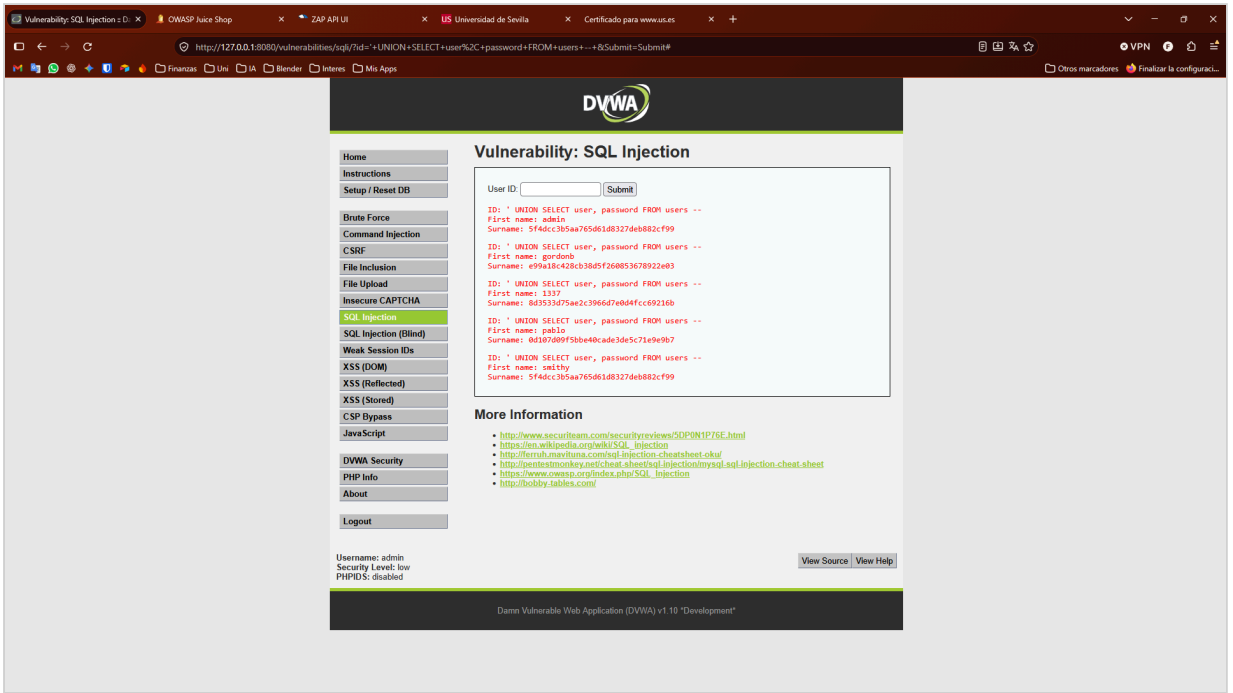


Figura 6.9: Volcado completo de la tabla de usuarios mediante inyección UNION, mostrando los nombres de usuario y sus hashes MD5 de contraseña.

2. Cross-Site Scripting reflejado y almacenado (A05 – Injection)

En el módulo *XSS (Reflected)*, la carga `<script>alert(1)</script>` introducida en el campo de entrada se ejecuta directamente en la respuesta del servidor. En el módulo *XSS (Stored)*, la carga `<img src=x onerror=alert(1)>` se almacena de forma persistente en el libro de visitas y se ejecuta en cada carga de la página, incluso tras recargarla, afectando por tanto a cualquier usuario que la visite. El

Fuzzer alimentado con `xss.txt` permite determinar qué variantes atraviesan los filtros sin ser saneadas.

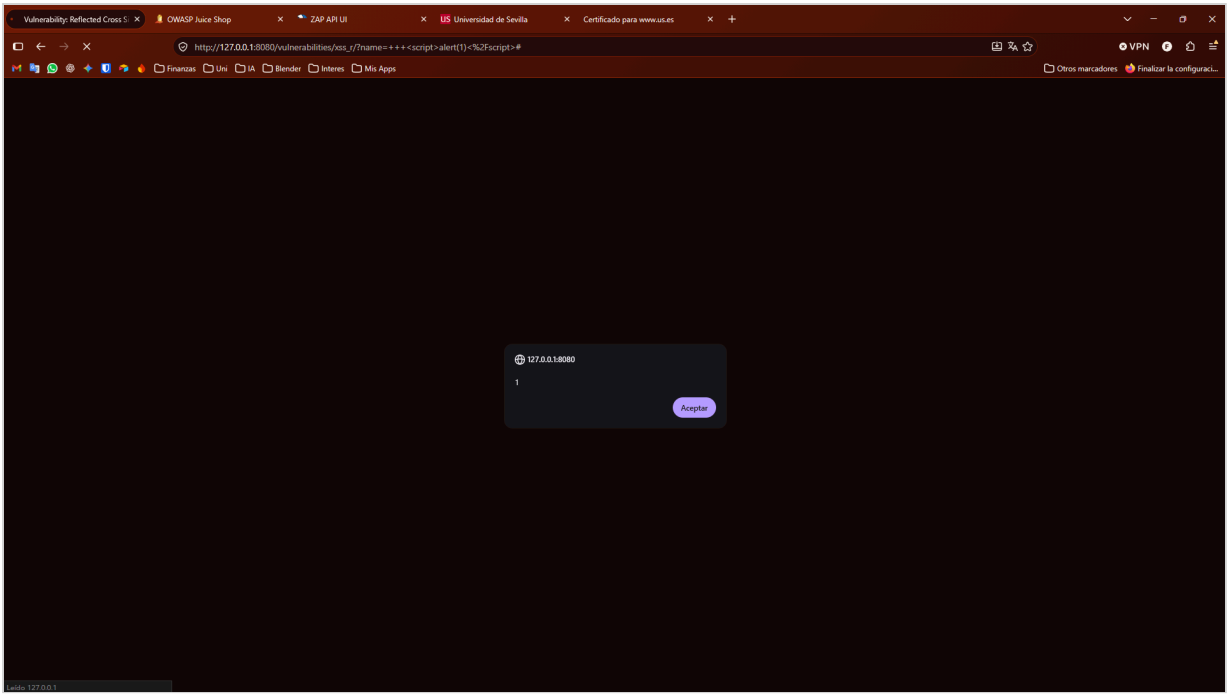


Figura 6.10: Ejecución de la carga de XSS reflejado introducida en el parámetro de entrada de la aplicación.

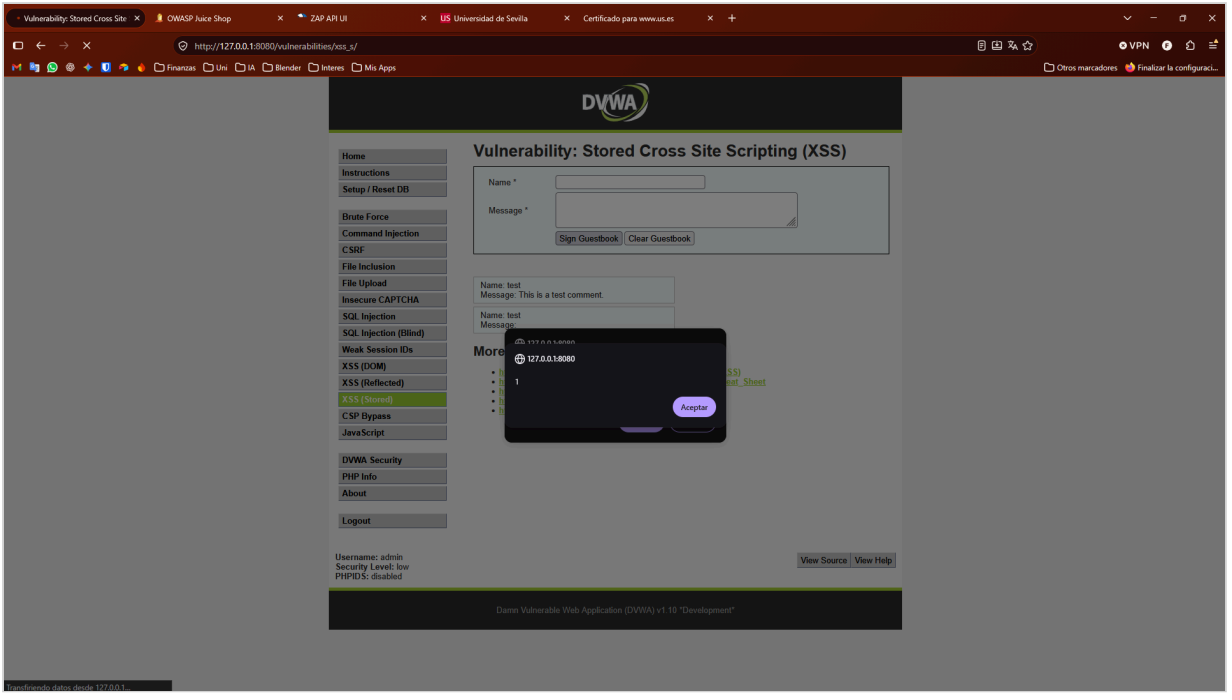


Figura 6.11: Ejecución de la carga de XSS almacenado tras recargar la página, evidenciando su persistencia en la base de datos.

3. Path Traversal / File Inclusion (A01 – Broken Access Control)

Sobre el parámetro `page` del módulo *File Inclusion* de DVWA, la petición `?page=/etc/passwd` mediante ruta absoluta devuelve el contenido íntegro del fichero `/etc/passwd` del contenedor,

incluyendo entradas como `root:x:0:0:...` y `mysql:x:101:101:...`. El diccionario `path-traversal.txt` contempla adicionalmente variantes de ruta relativa y codificada (`../`, `..%2f`). El hallazgo demuestra la posibilidad de lectura de ficheros arbitrarios del servidor.

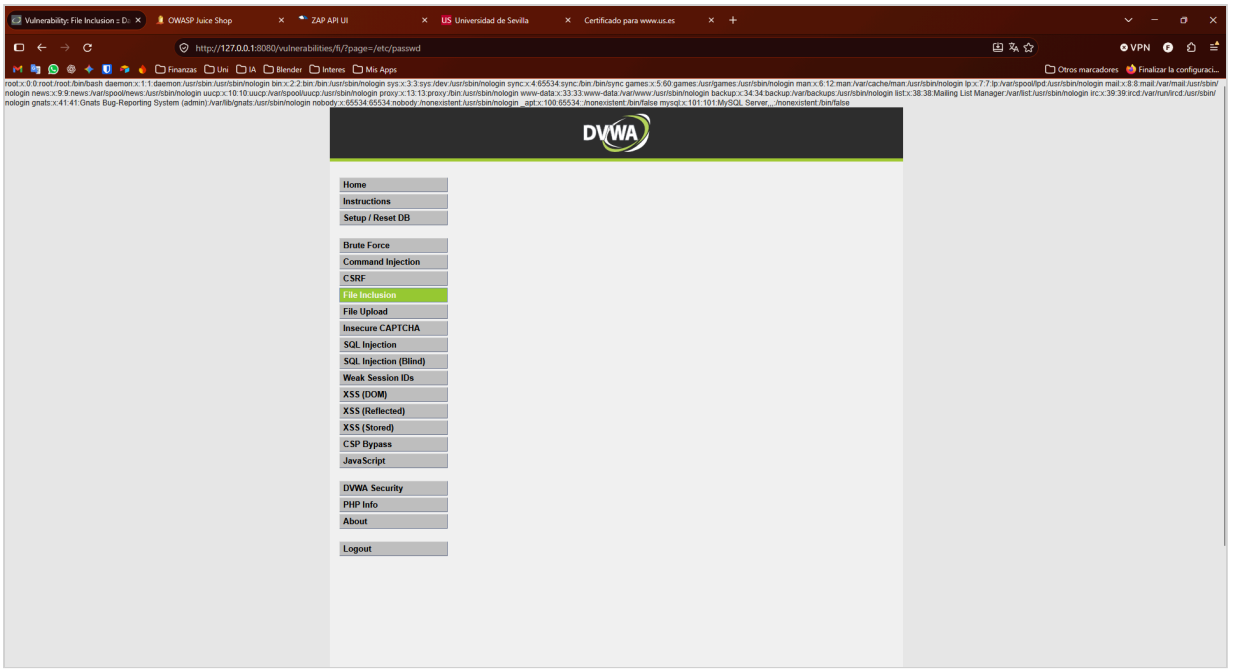


Figura 6.12: Lectura del fichero /etc/passwd del servidor a través de la inclusión de ficheros mediante ruta absoluta en el parámetro page.

Escaneo activo y recopilación de evidencias

Finalmente se ejecutó un rastreo (*Spider*) y un escaneo activo (*Active Scan*) completos sobre `http://127.0.0.1:8080`, exportándose el informe en formato HTML, los resultados del Fuzzer en formato CSV y la sesión de ZAP. El escaneo generó un total de 37 alertas, entre las que destacan la ausencia de tokens anti-CSRF, una política de seguridad de contenido (CSP) no configurada o con directivas débiles, la exploración de directorios, la reescritura del identificador de sesión en la URL, la presencia de una librería JavaScript vulnerable y la ausencia de los atributos *HttpOnly*, *Secure* y *SameSite* en las cookies, elementos clasificables bajo la categoría A02 – Security Misconfiguration.

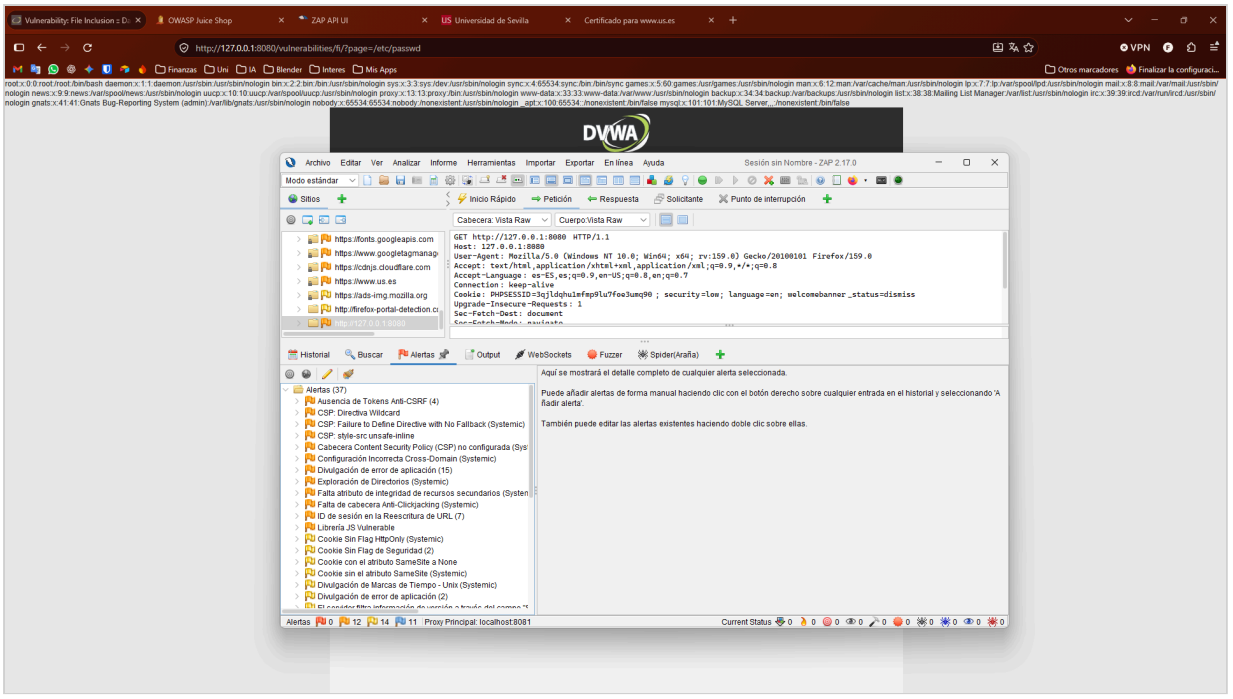


Figura 6.13: Panel de alertas de ZAP con las 37 incidencias detectadas durante el escaneo activo de la aplicación.

Plan de mitigación

Para cada vulnerabilidad identificada se propone la contramedida estándar correspondiente, cuya eficacia se verificó repitiendo las mismas pruebas con la aplicación configurada en nivel *high/impossible*:

Vulnerabilidad	OWASP 2025	Mitigación propuesta
Inyección SQL	A05 – Injection	Uso de consultas parametrizadas (*prepared statements*), principio de mínimo privilegio en la cuenta de base de datos y supresión de la concatenación de entrada del usuario.
XSS reflejado y almacenado	A05 – Injection	Codificación contextual de la salida, validación de la entrada, política de seguridad de contenido (CSP) y cookies con atributo HttpOnly.
Path Traversal / LFI	A01 – Broken Access Control	Lista blanca de ficheros permitidos, canonicalización y validación de rutas y restricción del directorio base.
Cabeceras de seguridad ausentes	A02 – Security Misconfiguration	Implantación de HSTS, CSP, X-Frame-Options y X-Content-Type-Options.

6.3. Conclusiones

La auditoría dinámica confirmó que el proxy interceptor permite trazar de forma íntegra el tráfico HTTP y HTTPS sin alterarlo, gracias a la importación de la autoridad raíz de ZAP en el navegador, así como operar como intermediario activo cuando se habilita la interceptación. Sobre la aplicación de laboratorio se verificaron tres vulnerabilidades explotables del OWASP Top 10 2025 —inyección en sus

variantes SQL y *Cross-Site Scripting*, y control de acceso roto a través de *Path Traversal*—, todas ellas con evidencia reproducible mediante el Fuzzer y comprobación manual, complementadas por las 37 alertas del escaneo activo.

La verificación de las contramedidas elevando el nivel de seguridad de la propia aplicación confirma que el origen de las vulnerabilidades reside en una validación de entrada y una codificación de salida deficientes, y no en la plataforma subyacente. Las mitigaciones propuestas son contramedidas estándar, directamente aplicables y verificables, que cierran los vectores de ataque identificados durante la auditoría.